Your Code

```python
students = [
("Arun",[78,85,92,67]), ("Bala",[45,67,55,49]),
("Charan",[90,92,88,95]), ("Divya",[65,72,70,68]),
("Esha",[88,76,91,84])
]

a = {n:sum(m)/4 for n,m in students}
t = {n:sum(m) for n,m in students}

print("Totals:", t)
print("Averages:", a)
print("Highest Total:", max(t,key=t.get))
print("Lowest Average:", min(a,key=a.get))

ca = sum(a.values())/5
print("Above Class Average:", [n for n in a if a[n]>ca])
print("3+ above 80:", [n for n,m in students if sum(x>80 for x in m)>=3
print("Subject Highest:", [max(m[i] for n,m in students) for i in range
print("Below 50:", [n for n,m in students if min(m)<50])

s = sorted(students,key=lambda x:sum(x[1])/4,reverse=True)
print("Sorted:", [(n,a[n]) for n,m in s])
print("Top 2:", [n for n,m in s[:2]])
```

Input

```
stdin input
```

Output

```
Totals: {'Arun': 322, 'Bala':
216, 'Charan': 365, 'Divya': 275,
'Esha': 339}
Averages: {'Arun': 80.5, 'Bala':
54.0, 'Charan': 91.25, 'Divya':
68.75, 'Esha': 84.75}
```

Your Code

```python
import string

text = """Python programming is easy to learn.
Python programming is powerful.
Learning Python requires practice.
Practice makes programming skills better."""

text = text.lower().translate(str.maketrans("", "", string.punctuation)
words = text.split()
unique = set(words)

print("Lowercase:", text)
print("Words:", words)
print("Total Words:", len(words))
print("Unique Words:", len(unique))
print("Frequency of python:", words.count("python"))
print("Frequency of programming:", words.count("programming"))
print("Repeated Words:", [w for w in unique if words.count(w) > 1])
print("Longest Word:", max(words, key=len))
print("Sorted:", sorted(unique))
print("Words containing 'p':", [w for w in unique if "p" in w])
print("Percentage of Unique Words:", len(unique)/len(words)*100)
```

Input

```
stdin input
```

Output

```
Lowercase: python programming is
easy to learn
python programming is powerful
learning python requires practice
practice makes programming skills
better
```

SIMATS Online Programming Lab

PYTHON PROGRAMMING

SAFRIN RITHA AM
192313059RC

Arun, Charan, Esha

Your Code

```
employees = [
("E101","Arun",["Python","SQL","ML"]),
("E102","Bala",["Java","SQL","HTML"]),
("E103","Charan",["Python","ML","SQL"]),
("E104","Divya",["Python","Java","Cloud"]),
("E105","Esha",["Python","ML","Cloud"])
]

print("Python:",[n for i,n,s in employees if "Python" in s])
print("Python + ML:",[n for i,n,s in employees if "Python" in s and "ML
print("SQL but not Java:",[n for i,n,s in employees if "SQL" in s and "
print("Java or Cloud:",[n for i,n,s in employees if "Java" in s or "Clo

skills = {x for i,n,s in employees for x in s}
print("Skill Count:",{x:sum(x in s for i,n,s in employees) for x in ski
print("Most Common Skill:",max(skills,key=lambda x:sum(x in s for i,n,s
print("3 Skills:",[n for i,n,s in employees if len(s)>=3])
print("Without Python:",[n for i,n,s in employees if "Python" not in s]
print("Sorted:",sorted(employees,key=lambda x:len(x[2])))
print("Suitable:",[n for i,n,s in employees if "Python" in s and "ML" i
```

Input

stdin input

Output

```
Python: ['Arun', 'Charan',
'Divya', 'Esha']
Python + ML: ['Arun', 'Charan',
'Esha']
SQL but not Java: ['Arun',
'Charan']
```

SIMATS Online Programming Lab

PYTHON PROGRAMMING

SAFRIN RITHA AM
192313059RC

Yes

Your Code

```
ees = [
","Arun",["Python","SQL","ML"]),
","Bala",["Java","SQL","HTML"]),
","Charan",["Python","ML","SQL"]),
","Divya",["Python","Java","Cloud"]),
","Esha",["Python","ML","Cloud"])


"Python:",[n for i,n,s in employees if "Python" in s])
"Python + ML:",[n for i,n,s in employees if "Python" in s and "ML" in s
"SQL but not Java:",[n for i,n,s in employees if "SQL" in s and "Java" 
"Java or Cloud:",[n for i,n,s in employees if "Java" in s or "Cloud" in

 = {x for i,n,s in employees for x in s}
"Skill Count:",{x:sum(x in s for i,n,s in employees) for x in skills})
"Most Common Skill:",max(skills,key=lambda x:sum(x in s for i,n,s in em
"3 Skills:",[n for i,n,s in employees if len(s)>=3])
"Without Python:",[n for i,n,s in employees if "Python" not in s])
"Sorted:",sorted(employees,key=lambda x:len(x[2])))
"Suitable:",[n for i,n,s in employees if "Python" in s and "ML" in s])
```

Input

stdin input

Output

```
Python: ['Arun', 'Charan',
'Divya', 'Esha']
Python + ML: ['Arun', 'Charan',
'Esha']
SQL but not Java: ['Arun',
'Charan']
```

```python
    if all(conditions):
        valid.append(password)
    else:
        invalid.append(password)

print("Valid:")
for password in valid:
    print(password)

print("\nInvalid:")

for password in invalid:
    failed = []

    if len(password) < 8:
        failed.append("Less than 8 characters")

    if not any(c.isupper() for c in password):
        failed.append("No uppercase")

    if not any(c.islower() for c in password):
        failed.append("No lowercase")

    if not any(c.isdigit() for c in password):
        failed.append("No digit")

```

SIMATS Online Programming Lab

PYTHON PROGRAMMING

SAFRIN RITHA AM
192313059RC

Expected Output:
['arun', 'bala', 'charan']

Your Code

```python
records = [
    ("Arun","Python"),("ARUN","Python"),
    ("Bala","Java"),("bala","Java"),
    ("Charan","Python")
]

names = []

for record in records:
    name = record[0].strip().lower()
    if name not in names:
        names.append(name)

print(names)
```

Input

stdin input

Output

Longest Word:
visualization

Your Code

```
text = "data science uses data analysis and data visualization"
w = text.split()
u = list(dict.fromkeys(w))
f = {x:w.count(x) for x in u}

print("Words:", w)
print("Total:", len(w))
print("Unique:", u)
print("Frequency:", f)
print("Most Frequent:", max(f,key=f.get))
print("Repeated:", [x for x in u if f[x]>1])
print("Tuples:", [(x,f[x]) for x in u])
print("Sorted:", sorted([(x,f[x]) for x in u],key=lambda x:x[1]))
print("Frequency >1:", [x for x in u if f[x]>1])
print("Longest:", max(w,key=len))
print("Starting d:", [x for x in u if x.startswith("d")])
print("Repeated %:", len([x for x in w if f[x]>1])/len(w)*100)
```

Input

stdin input

Output

```
Words: ['data', 'science',
'uses', 'data', 'analysis',
'and', 'data', 'visualization']
Total: 8
Unique: ['data', 'science',
'uses', 'analysis', 'and',
```

SIMATS Online Programming Lab

PYTHON PROGRAMMING

SAFRIN RITHA AM
192313059RC

Your Code

```
students = [
    ("A001", "ARUN KUMAR", [78, 85, 92]),
    ("A002", "Bala", [45, 67, 52]),
    ("a003", "arun kumar", [88, 91, 95]),
    ("A004", "CHARAN", [72, 76, 81]),
    ("A005", "Divya", [55, 48, 62])
]

records = []

for sid, name, marks in students:
    sid = sid.strip().upper()
    name = name.strip().lower()
    records.append((sid, name, marks))

print("Normalized Records:")
for sid, name, marks in records:
    print(sid, name, marks)

totals = {}
averages = {}

for sid, name, marks in records:
    total = sum(marks)
    avg = total / len(marks)
```

Input

stdin input

Output

```
Traceback (most recent call
last):
  File
"C:\Windows\TEMP\sandbox_19231305
9RC_6a8fcdbd1f6bb7.35493418\main.
py", line 31, in <module>
```

```python
for sid, name, marks in records:
    total = sum(marks)
    avg = total / len(marks)
    totals[name] = total
    averages[name] = avg

print("\nTotals:")
for name, total in totals.items():
    print(name, "→", total)

print("\nAverages:")
for name, avg in averages.items():
    print(name, "→", f"{avg:.2f}")

passed_all = []

for sid, name, marks in records:
    if all(mark >= 50 for mark in marks):
        passed_all.append(name)
```

```python
    print(name)

failed_one = []

for sid, name, marks in records:
    if any(mark < 50 for mark in marks):
        failed_one.append(name)

print("\nFailed At Least One:")
for name in failed_one:
    print(name)

highest_total = max(totals.values())

print("\nHighest Total:")
for name, total in totals.items():
    if total == highest_total:
        print(name, "→", total)

lowest_total = min(totals.values())

print("\nLowest Total:")
for name, total in totals.items():
```

Your Code

```python
names = [
" Arun Kumar", "ARUN KUMAR", " arun kumar",
"Bala Krishnan", "BALA KRISHNAN",
"Charan", "charan", "DIVYA DEVI", "Divya Devi"
]

n = [x.strip().lower() for x in names]
u = list(dict.fromkeys(n))

print("Normalized:", n)
print("Unique Customers:", u)
print("Number of Unique Customers:", len(u))

print("Duplicate Customers:", [x for x in u if n.count(x) > 1])
print("Names with Multiple Words:", [x for x in u if " " in x])
print("Longest Name:", max(u, key=len))
print("Starts with A or D:", [x for x in u if x[0] in "ad"])
print("Characters:", {x:len(x) for x in u})
print("Sorted:", sorted(u))
print("Most Frequent:", max(u, key=n.count))
```

Input

```
stdin input
```

Output

```
Normalized: ['arun kumar', 'arun
kumar', 'arun kumar', 'bala
krishnan', 'bala krishnan',
'charan', 'charan', 'divya devi',
'divya devi']
Unique Customers: ['arun kumar',
```

our Code

```python
passwords = [
    "Python@123",
    "python123",
    "PYTHON@123",
    "Py@45",
    "Python123",
    "Java#4567",
    "Admin@2026"
]

valid = []
invalid = []

for password in passwords:
    conditions = []

    conditions.append(len(password) >= 8)
    conditions.append(any(c.isupper() for c in password))
    conditions.append(any(c.islower() for c in password))
    conditions.append(any(c.isdigit() for c in password))
    conditions.append(any(not c.isalnum() for c in password))

```

Input

```
stdin input
```

Output

```
Traceback (most recent call
last):
  File
"C:\Windows\TEMP\sandbox_19231305
9RC_6a8fcd3a97e681.91325806\main.
py", line 52, in <module>
```